dib BASIC manual

An Independent Project on the emulation of the Dartmouth Time-Sharing System - the BASIC emulator

Antonio Maschio

Independent Project

25 July 2026

ABSTRACT

Introducing the dib BASIC emulator of the 1966 BASIC version III as was available in the Dartmouth Time-Sharing System.

This text is not a BASIC primer nor a BASIC manual; I assume you have the most common knowledge about protgramming, BASIC and the DTSS; this text is more like a reference document that helps in using correctly the various features, their limits and enhancements.

Even though the bulk of the work is mine, it's undeniable that the Dartmouth documents available at `bitsavers.org` were of immense utility, and constitute the guide of this work.

Nonetheless, coding is my sole responsibility, so any error in the code is due to me and me only. Readers are encouraged to report all errors and inconsistencies (both in the program and in this text) to:

`ing dot antonio dot maschio at gmail dot com`

Support for my efforts is appreciated. Donations can be made via PayPal to

`tbin at libero dot it`

Thanks.

Non disse Cristo al suo primo convento:
'Andate, e predicate al mondo ciance';
ma diede lor verace fondamento;
(Dante, Paradiso, Canto XXIX, 1472)

Christ said not to his first college:
´Go forth, and preach idle stories to the world';
but gave to them a foundation for the truth.
(Dante, Paradise, Chant XXIX,
transl. Courtney Langdon, 1920)

1. A FOREWORD

Sometimes I read on some mailing list about programming, or on newsgroups dealing with this or that programming language, or on clever programmers' web pages, that "*BASIC was a primitive language*"; well, this may be true, in some arcane sense, especially if you consider some modern complicated language like java, C++ or Delphi, or peculiar languages such as Prolog, Lisp, Forth. But I contest this sentence

1) because BASIC '*was*' not, BASIC '*is*';

2) because BASIC is not primitive: it's *simple*. It's a rather different question.

Imagine you have to solve a math problem, say a system of linear equations, and suppose you've got to write a program for this; which language will you choose? Your '*favorite*' may be too big for such task. Well, in the best case you know this language very well, and you will end up without spending hours in writing the program, but in the worst case you're lucky if you have not to plunge into a 1000 pages user manual or learn tens of system calls. Why not using Dartmouth BASIC? No more than a bunch of lines (containing even the problem data) and the solution is there. No manuals, no system calls.

Yes, BASIC is an unstructured language - and someone said that a programmer exposed to BASIC is likely to never learn well another language (this is not true, obviously) - so I guess that, if BASIC is unstructured, it expects you to give it a structure. And the structure you create may make it a spaghetti-code listing, or you may make a superb listing of it. Dartmouth BASIC gives you two dozens of statements and a consistent way to use them. It gives you a simple structure for each statement and **you** have to build a smart structure for your program. It's up to you: BASIC has no soul, it waits for you to provide one.

After all, BASIC is a language that can be used by anyone, that won't force you to format numbers output, that uses matrices like numbers, that solves systems of equations in a bunch of statements, that remains familiar and usable for years. Some of us couldn't ask for more.

Yes, using such a language today is quite unrealistic. I had to write my own interpreter to do it, and this is not what I call a friendly way. But hail to those guys who conceived such a language in 1964!

2. A BIT OF PERSONAL HISTORY AS INTRODUCTION

It was during my High School days that I met the BASIC language for the first time; it was a schoolmate, who had later to become my best friend, to introduce me to programming, because of his wonderful brand new Commodore 64, he received for 1982 or 1983 Christmas (memories fade). From this moment on, I kept trying to convince my parents of the extraordinary need I had for a computer like that one and that I couldn't even imagine how to get through University without it! Lost words. My father, a very practical man, whose sight was very long, told me he could get through his High School degree with a 5 inches slide-rule (imagine I asked: what the devil *is* a slide-rule?) Thus, a computer was not, by any definition you could give for it, a 'need'. Question closed.

What could I do? I spent a lot of time in my friend's room, staring at the wonderful (256) colors of the screen, and learning my first BASIC words while listening to him reading sources and watching him typing commands. Once back home, the only way to relax was playing my guitar, because I had no computer at home, and even my calculator was not programmable. I was in a sort of "need of programming"... Anyway, for my next birthday, my father, who was a very practical man but also capable of extraordinary outbursts of generosity, he surprised me, and made me the gift I wanted: the CBM64! I was a bit surprised, because I think I had stressed him a lot for that but I thought he could resist another couple of years.

In Italy, those were the days of microcomputers, and first computer magazines - BIT, Microcomputer, and others, - wonderful glossy magazines, full of listings in BASIC or other languages for this or that computer, fighting a sort of battle of the dwarves between the CBM64 and the ZX Spectrum. I was a 64er.

During my third High School year (scientific branch) I was studying my first math function in the Cartesian space, so it was natural joining this subject with the extraordinary abilities of the C64; the first thing I wrote was a program to draw a function y=f(x) on the screen, plotted in high resolution (I'm afraid I was not so smart to do it all alone: probably, I took the original source from a magazine and changed it a bit). Anyway, the pride I felt showing the results to my father, who stared many minutes at the image, imagining what future was waiting for me, pushed me to study programming more seriouisly; I then bought the C64 monitor, the 1541 disk drive and the book "Commodore 64 C= - Reference Guide for the Programmer" [4].

I began studying [4], learning that machine language was available through some hacking, learning about sprites, the three independent music oscillators (a system called SID), and all the peculiar memory addresses, but I didn't go beyond BASIC for two reasons: first, machine language seemed to me too difficult and strange; and second, I thought BASIC was a wonderful language to do quite everything (though programs ran slowly!). I bought then the book "Programming with BASIC", by Byron S. Gottfried [3] and began studying it seriously. The book was very professional, and all (or almost) was applicable to my home computer, so I felt like a very mature programmer! One thing baffled me: the fact that all MAT statementsm, that were part of the compiler used by Mr. Gottfried, were completely missing from my home computer, but I was sure it was more recent than the one described in the book!

Anyway, in 1985 I began my University years, and for many years I couldn't do anything else with the Commodore unit. I studied Pascal and Fortran 77, using the VAX mainframe of the University Calculus Room, equipped with UNIX (circa 1987-88); much later, I could use PCs with DOS in the computer room of the Civil Engineering Faculty (circa 1990-91, I believe it was DOS version 2.x), and so the CBM64 was left behind, with all its power. At home, abandoned, the C64 went ruined, and one day my mother got rid of it without even telling me. The awful day is carved in my memory, and I was very sad that day!

In 1993 my parents bought me a PC with Windows 3.11: it served me to do some word processing for my thesis, but it was also a starting point in serious computing. When I finally got my degree in Civil Engineering, in 1994, I began my job, and years began to fly (yes, none years to get a degree...).

In 1998 I installed linux on my PC and began studying C, bash and prolog, and BASIC was not in my mind. So I lost any contact with it: line numbering, GOTOs, the absence of specific functions and procedure calls, and other minor things, confronted to the "new" languages I was learning, led me to forget BASIC, because it was old.

*****

Time takes its time to work, but it works. In the summer of 2004, I visited Ron Nicholson's site by chance, and discovered his chipmunk basic interpreter. Ah! BASIC was not dead, after all!

Next, reading some documents found in Internet (the "Dartmouth BASIC manual" for version II, October 1964 [1]

in particular) I learned that BASIC was born in the Dartmouth College in 1964 (many years before C!), that its first versions were real compilers and that they dealt naturally with matrices (a feature that was stripped from all subsequent versions of BASICs). These facts were so appalling that I took the C64 manual and the Gottfried book out of their dust; reading them again revealed me one mere thing: I didn't cease to love BASIC.

Next step was conceiving a BASIC interpreter on my own, to be developed on my brand new emac 1.42 equipped with Mac OS X. Why not? There are dozens of compilers and interpreters in the world, one more will do no harm. The reason that led me to forget BASIC is now the reason that leads me to love it: because it's old!

This love never ceased. I built, after dib, also decb, an interpreter for the DEC-20 BASIC, and tbas, a general purpose structured BASIC. I changed computer (now I'm using a Linux box) But, you bet? dib is my favourite, because it's the first...

I can see, in the code, all the expertise that now I have and then I had not. Naive is not the right term. Crude, I'd say. But it works now as it worked then, and with a modern gcc (version 15, which is a monster with respect to the first compilation, made with version 2.95) it compiles without a warning. This must mean something.

3. THE INTERPRETER

In the following are exposed all the characteristic of the BASIC interpreter. In this text, the BASIC language is not exposed, if not briefly. You should refer to other texts in the package (and in literature) for news about the 1966 BASIC syntax.

3.1. The available options

When invoked, dib must be invoked with the following syntax:

```
$ dib [options] file
```

Options are:

```
-c                Continue execution in case of non-blocking errors
    --conditions  Print redistribution conditions from GPL3
-d                Activate debug mode
    --errors-list Print a complete errors list and exit
-h, --help        Display this information and exit
-i                Display program header
-l                List the program <file> and exit
-p 'expr'         Print BASIC expression 'expr' and exit
-s<n>             Start executing BASIC program from line <n>
-t                Print a time report after execution(s)
-v, --version     Display version and exit
    --warranty    Print warranty conditions from GPL3
-z                Print as zero numbers smaller than 1E-10
```

A version of this brief list appears when dib is invoked as `dib -h`/`dib --help`.

`file` is a textual file with BASIC code; preferred extension (implicit) is `.bas`.

☞ **Note**: all options that perform a task and then cause dib to exit, without any file elaboration, are generally unaware of the existence of other options specified on the same command line after them; thus, these options should be used alone, and they generally don't need a file name to operate.

Here are all options detailed:

-c continue; after an error, generally, dib stops and prints a message; if you want it to go on, activate this option, but ***at your risk***! In fact, the program may call not instantiated variables, or execute jumps or math expressions with values not correctly initialized, so a system error (like "segmentation fault" or "BUS error") may appear onto you terminal. See paragraph <3.5>.

-d activate debug mode; when debug mode is on, the current line is printed before it is executed; this of course alters the output behavior, but you can trace the program flow, and maybe understand why your BASIC code is not working.

-h print the standard UNIX help and exit (also `--help`).

-i print the program header; the header looks like this:

```
 PROG24    18:37   JUN. 26, 2026
```

and it's printed before any output of the program. This can be used to document the output. If run within deer this option is not necessary, because deer prints its own header.

-l list the argument file with 5 digits for the line number and a space, followed by the program line. This trick aligns all lines and gives a very professional look to the listing. E.g.

```
 00005 PRINT "EXAMPLE OF MAT STATEMENTS USAGE"
 00010 DIM A(2,2),B(2,2),C(2,2)
 00015 DIM D(2,2)
 00020 DATA 1,2,3,4,5,6,7,8,9
 00030 DATA 2,3,4,5,-6,7,8,-9,-10
 00040 MAT READ A,B
 00050 PRINT "MATRIX A"
 00060 MAT PRINT A
```

-p print a BASIC expression and exit; this option must be followed by a BASIC expression enclosed in single quotes; by the term `<basic expression>` I mean anything syntactically suitable for the `PRINT` statement: strings, calculations, mixed values; of course, variables and matrices cannot be used in this context (it would be a nonsense, since they cannot be initialized within a `PRINT` statement). So, from the command line, invoking:

```
$ dib -p 'expr'
```

is equivalent to the BASIC statement

```
PRINT 'expr'.
```

All functions in capitals, of course! The space between `-p` and the first single quote is optional.

-sn This option enables dib to start a program from line number `n`, rather than from the first available only; I don't know if the original Dartmouth Operating System could do this, but I guess it's a minor point.
If the starting line specified in the -s option does not exist, the first valid forward line in the source code will be the first available; e.g.: if you have a source in file `PROG24` with code lines between 10-70 and between 100-150, by invoking:

```
$ dib -s80 PROG
```

will make the program start execution at line 100, since line 80 doesn't exist. No space is allowed between -s and the number, and if the number is zero or negative, it is automatically promoted to 1.

-t after the program is terminated, print a timer report in the form:

```
TIME:   N SECS.
```

where `N` is the time the execution took. It is in the style of the 1966 Dartmouth BASIC manual (see for instance p. 5) and measure to MSEC to HOUR times. In deer this option is not necessary, because deer prints its own timing string after the execution of a BASIC program.
Options -i and -t can be used together to give the output of the DTSS operating system, where the original BASIC version III ran.

-v print the standard UNIX version banner, and exit (also `--version`).

-z (portrait zero) All numbers lower than 1E-10 are printed as zero, useful when they may clutter the output, e.g. with matrices. In this case, you can distinguish "pure" zeroes from these approximations because the latter are always preceded by their sign, positive or negative; so that: 0 means a zero as calculated by dib, -0 means an approximation downward (for instance -2.3E-14), +0 means an approximation upward (for instance 2.3E-14); you can change the limit of 1E-10 changing the EPSILON constant in `dib.h`, but a valid number must be greater than `ZERO` (see par. 3.3).

To know terms of warranty and conditions, type '`dib --warranty`' and '`dib --conditions`' (these commands print extracts of license, as required by the GPL v3).

3.2. Customizing the runtime phase

In the file `dib.h` there are some values you can safely change to seize dib to your needs:

`ef` is the variable flag that controls the extended features activation; it's `FALSE` by default; you can turn it to `TRUE` to have the extended features activated from start; if you do it, using option `-x` will disable extended features, rather than enable them.

false_
(mind the final underscore) is the flag that controls error short-circuit; by default it's `TRUE`, which means that dib stops at every error met; if you set it to `FALSE`, dib will stop only when a blocking error is met (see ahead); if you do it, option `-c` will set `false_` to `TRUE` rather than setting it to FALSE.

`EPSILON`
is the limit for numbers to be printed as zero when option `-z` is invoked. By default it is `1E-10`, so that all numbers lower than `EPSILON` are printed as zero with option `-z`; you can put here any value you like, provided it's not smaller than `ZERO`, the lowest number dib can read (`4.31808427754E-78`): smaller values are for dib undistinguishable from zero.

Don't change other variables, parameters, constants, unless you know very well what's that you're doing: you may mess up things and end up with an useless program.

3.3. A datum is a datum is a datum...

The BASIC elements are exposed in the following.

3.3.1. Numbers

Numbers range has the following limits:

- the max (positive) real is 5.78960446187E76, identified as `INFF`
- the min (negative) real is -5.78960446187E76, identified as `MINFF`
- the smallest real absolute value is 4.31808427754E-78, identified as `ZERO`. These values have been set after reading the notes about the `OVERFLOW` and `UNDERFLOW` errors on [12], page 37. Difference are small, though, and I consider these new values only a refinement. (See ahead, errors #30/31.)

There's more to say: the `INFF/MINFF/ZERO` limits are not set into values themselves, but only in printing; so the internal value is preserved for further calculations

As the original BASIC, dib deals with a single number type: real. Any number is written as an integer if the internal representation of the real is made of all zeroes after the dot; e.g.:

```
10 PRINT 1.0
```

will print

```
 1
```

Notice the space before the number: a place for the sign is always printed in Dartmouth BASIC and, if positive, a space - not a plus - is printed. Sometimes reals behave like integers if they have too many zeroes after the dot and before a non-zero digit; e.g.:

```
10 PRINT 1.0000002
```

will print

```
 1.
```

Notice the dot after the number, meaning that the number is not really an integer, but a "masked" real. To this regard, I must observe that, due to the different environments (the GE mainframe for the Dartmouth BASIC, the UNIX gcc for dib), a number is not always pictured in the same way; for instance, dib is much more precise, and integers numbers are printed as integers (without the dot, as it should be), while the original BASIC carried into the numbers a small approximation, causing them to be printed as integers with the dot; this affects the output (one more character for each number, after all), and sometimes - above all with the `MAT PRINT` statement - this means the output of the two versions coud differ.

If the whole number lesser than 1 may be written within six digits after the dot without approximation, it's written "as is"; for instance:

```
10 PRINT 0.0047
```

will print

```
 .0047
```

Any number may be input using the ten digits, the prepended minus, the dot and the letter `E` for powers of ten (`.2` is accepted, `1E2` too, `E2` is not). Follow instructions on the 1966 manual at page 7: dib is built on the same basics.

According to [11], the use of booleans was not in the Original Dartmouth BASIC; dib, on the contrary, treats any expression as a truth value, since truth values are simply real numbers (set as `FALSE=0` and `TRUE<>0`); of course, this is meaningful only if you print out a truth value, e.g. `PRINT 3>2` (which is quite useless on its own) or if you use a variable as truth value, e.g. `IF A THEN` (which was, probably, not available in the original BASIC); in the first case the output will be `-1`, in the second case, the part after `THEN` will be executed if `A` is different than 0 (any value). Remember that this may not be available in other dialects.

The 1964 manual, at page 8, adds: "*It should be noted that .000123456789 is illegal, and must be written as, say, .123456789E-3*"; this is not true for dib, which accept also that "illegal" form. Moreover, this program, made in C,

offers great precision in calculation, probably greater than the original BASIC; the program that calculates the powers of integers from 1 to 7 in [1] at page 39, prints calculated numbers with a final dot, while the input values are printed as pure integers. The same program run with dib prints all numbers as pure integers (as they actually are).

3.3.2. Variables

dib uses variable names as the original BASIC, nothing more. So `A..Z` and `A0..Z9` are the ranges of all the available variable names. Double letters or even longer names are forbidden - e.g. `AB` or `TANKVOLUME` (but see next subparagraph called "Capitals").

dib, as a safety measure, initializes all variables to zero (arrays included), but this feature was in Dartmouth BASIC only starting from version IV of 1967 (see [2]).

There are no string variables. This makes the Dartmouth BASIC a mathematic-oriented tool.

3.3.3. Arrays

Arrays in dib, as in the original BASIC, may have one or two dimensions; one for vectors, two for matrices (the original 1966 manual called them respectively lists and tables). The `DIM` statement instantiates the necessary memory space for the array:

```
10 DIM A(23),B(14,5)
```

and any array element is accessed through the same interface:

```
20 LET Y=A(2)+B(2,1)
```

If you use an array without `DIM`ensioning it, it will be automatically instantiated to a 10 elements vector or to a 10x10 matrix, i.e.:

```
10 LET Y=C(2)
```

will implicitly instantiate C as vector, while

```
20 LET X=D(3,6)
```

will implicitly instantiate D as matrix.

Of course, a reference to an undeclared array with any index greater than 10 will raise an error, independently of the memory space (don’t think that, since a 10x10 matrix may store at least 121 numbers, a call to A(15,5) will be accepted only because a 15x5 matrix may store at most 96 elements!). Note: the automatic instantiation is not taken into account within the `MAT` commands; when using a `MAT` command, you must priorly instantiate the necessary arrays (this is the same behavior, of course, of the original BASIC - see page 31 of the 1966 manual).

The original Dartmouth BASIC (from version IV on) had a strange behavior with array indexes: if used outside of the `MAT` environment, vectors and matrices had indexes starting from 0, while in the `MAT` environment they had indexes starting from 1. Not so for version III of 1966 of BASIC (that’s one of the reasons why I chose version III). dib is coherent, on this regard, as is version III: all indices start from 0. If you set option `-b` at start, though, or if you use the `OPTION BASE 1` statement in extended mode, indices will start from 1, both for vectors and matrices, into or out of the `MAT` environment.

*****

Vectors are always vertical; to declare a horizontal vector you have to use:

```
DIM A(0,N)
DIM A(1,N)
```

(the second in case of option `-b`); vectors, either declared with:

```
DIM Z(N)
```

```
DIM W(N,0)
```

behave in the same way with all commands, the latter being referenced as `W(N,0) and also as W(N)`.

☞ **Remember**: using option `-b` or `OPTION BASE 1` in extended mode, you cannot dimension vectors with indices starting from 0!

Arrays may be re-dimensioned, even using different depths, e.g. the following can be written into a file:

```
10 DIM A(6)
20 DIM A(4,4)
```

The latter will be the current declaration for the rest of the program. I don't know if this feature is common in other BASIC interpreters or compilers. In any case, the DTSS emulator by T. Kurtz admits this feature, and so does dib.

A final note: with `OPTION BASE 0` (default state), you may be tempted to dimension peculiar arrays like:

```
DIM A(0)
DIM A(0,0)
```

with the intention of creating a sort of degenerated one-element array. Well, this won't work, depending on the particular internal representation of arrays: such an array would be interpreted as a not instantiated array and thus re-dimensioned to a 10 elements vector or to a 10x10 matrix the first time it is used; any reference to `A(0)` or `A(0,0)` will of course be preserved, but beware when you use `MAT PRINT` on such arrays!

Arrays cannot contain strings. This is a consequence of the fact there are no string variables.

3.3.4. Precedence

Precedence among all the operators (not all available in default mode - see ahead), is set according to the following schema (higher to lower)

```
level 6: - (unary)
level 5: ()
level 4: ^ **
level 3: * /
level 2: + -
level 1: < > = <= >= <>
```

The symbol `**` may be used to indicate exponentiation.

The Original BASIC up to version II included had a bug, causing an expression like

```
-3^2
```

to yield 9 (as if `^` had a lower precedence than the unary minus); version III returned the correct result of -9) and so does dib, accordingly to version III, because `^` has a higher precedence than the unary minus.

3.3.5. Screen

A program line is 75 characters wide and so is screen output; anything beyond the 75th character in program lines will be lost; should they fall beyond the screen limit during output, numbers will be printed on a new line, strings will be cut.

The screen is to be seen as a printed paper flow. Screen columns are numbered 1 to 75, and this is quite different from the terminal (or terminal emulation) you are using, which is at least 80-characters wide. Anyway, this reflects the original input space of the Dartmouth BASIC, so you should never use program lines longer that 75 `*in all*` (line numbers included). Use an editor with characters-count like `vim`.

I took any care to simulate the output **exactly** as in the original BASIC. Some differences arise with calculation

results, because modern CPUs behave better than ancient ones...

3.3.6. Program lines formatting

A program line is a text line with the 1 to five digits at the start, followed by a single statement with its arguments. You can put any blank space into the line, even in the middle of a statement:

```
10 PRINT "HELLO, WORLD!"
10 PRINT   "HELLO, WORLD!"
10PR INT"HELLO, WORLD!"
10 P R I N      T "HELLO, WORLD!"
```

Previous lines are all equivalent, since all spaces/tabs are discarded during the line processing (of course, not for spaces/tabs into strings!). This said, you can format your listing to make it more readable, like:

```
10 FOR I=1 TO 10
20      PRINT I;
30      FOR J=1 TO I
40              PRINT "-->";I*J;
50      NEXT J
60 PRINT
70 NEXT I
80 END
```

without any risk of misinterpretation. If you want to use an editor with syntax coloring for BASIC, or use vim with the syntax file `dib.vim`, - see par. 3.7 - take care to write all statements with consecutive letters, in order to achieve the colored syntax: `PRINT` will be colored, `PR INT` will not (but will work).

Moreover, if you want to run your listings under different BASICs, be careful: they may not recognize broken statements.

I don't know if the original Dartmouth BASIC could accept broken statements like `PR INT` instead of `PRINT`; the DTSS emulator by Kurtz does, and so dib.

Program lines in a source file need not to be ordered: dib orders lines internally while loading the source. But remember that if a BASIC source file is unordered, it's unreadable for a human being.

Program lines may also be:

- void lines (or lines composed by spaces, tabs and blanks), which are simply discarded;
- lines with a line number only, which delete (if present) the stored line with that very line number; in fact, in case of two (or more) lines with the same line number, only the latter is maintained; you can use this trick to do some debugging; simply rewrite at the bottom of the file the lines to be substituted without deleting them really. Use a line number alone to temporarily disable a line.
- lines beginning with #; they are discarded too, to enable some BASIC scripting; simply copy the following line as the first line in the BASIC source file (this is not an original Dartmouth BASIC feature, of course), whose name, for this example, is `<filename>`:

  ```
  #!/path/to/dib <options>
  ```

  Put all the options you want. Then type from console:

  ```
  $ chmod +x <filename>
  ```

  and from now on you will have a sort of executable in BASIC.

3.3.7. Capital letters

All statements must be written in capitals; `PRINT` and print are not the same, the second raising an error, but dib, differently than the original BASIC, will gently accept lower letters into strings. The following will behave as

expected:

```
10 PRINT "UPPER LETTERS"
20 PRINT "lower letters"
```

There is even a further note, here. no check is done upon lower or upper letters for variables, and the variables memory space of dib is enough for all. So A and a are two distinct variables, A(1,1) and a(1,1) are two distinct arrays elements, FNA and FNa are two distinct functions! See the following example:

```
10 LET A=24
20 LET a=48
30 PRINT A
40 PRINT a
50 END
```

that has the following output:

```
24
48
```

Take this feature with care,

I) because this behavior differs from the original BASIC;

II) because it may make your listing incompatible for other BASIC interpreters and compilers

This said, you can count on twice the variables number of the original BASIC, twice the number of arrays, twice the number of functions... just in case you need it!

3.3.8. Pre-parsing

As said, dib is an interpreter, not a compiler. Nonetheless, some commands are pre-parsed before execution:

- END statements are checked in default mode for the following conditions: there must be only one END and it must be the last statement in the code; if it's not so, an error will be raised and the program won't be executed.

  In extended mode, no check is done for END, neither for being unique nor for being the last.

  Note: dib, as an emulator, has no need of an END statement; so, in this regard, the END keyword is dummy, i.e. it has no scope; in the original compiled BASIC it meant the end of the preparsing process, and had a very important role; when BASIC became interpreted (thanks, Bill!) it lost its utility, and END became redundant. dib restores its importance (at this point only historically) and classifies it as necessary. That's why it's pre-parsed.
- DATA statements are parsed to gather all the data information. Data are sequentially stored into a vector, and READ proceeds reading it from start. RESTORE resets vector pointer to the first element. Thus, during execution, DATA is a statement without effect, and is simply discarded.
- DEF FN structures are parsed to gather all functions and all the variables that are used as arguments. Successive calls to the DEF FN function force a parsing of the function expression after instantiating process variables. Thus, during execution, DEF is a statement without effect, and is simply discarded.

All errors that have some link with the pre-parsing phase occur even before the first program line is executed by Hdib.

3.3.9. Remarks

The syntax that must be observed, apart from inner spaces, is simple, as in the 1964 manual, and may be condensed into this terse sentence:

```
one statement per line
```

The colon to divide multiple instructions on one line is NOT permitted, and this reflects exactly the Dartmouth BASIC original philosophy (this is true for default mode, while extended features permit a limited use of multiple instructions for assignments).

Finally, I like the motto at page 21 of the 1964 manual (not repeated in the GE 1966 version, see [12]), here

reported:

TYPING IS NO SUBSTITUTE FOR THINKING

Many programmers (I'm the first) should learn from this.

3.4. The BASIC language

Here you will find the list of all dib's default statements and functions:

3.4.1. BASIC statements

Available statements are

- `PRINT`: display strings enclosed into double quotes, or expression built up from numbers, variables and operators; arguments may be followed by comma or semicolon; `PRINT` accepts even the classic string/value without comma or semicolon), whereas the comma pushes the next item to the next tab position (a tab position distance 15 characters from the previous), while the semicolon ignores the tab position regulation.
- `FOR..TO..STEP/NEXT`: execute cycles with a depth of 26 levels (the original basic admitted, for version II, 26 `FOR` cycles in total); `NEXT` must be followed by the relative variable name.
- `IF..THEN`: (followed by a valid address) executes jumps to a specific address if condition after `IF` is true.
- `GOTO`: jumps to any legal address (jumping out of a `FOR` cycle with `GOTO` or `IF..THEN` causes a cycle break); of course, `GO TO` is accepted.
- `GOSUB/RETURN`: (80 levels depth) jumps to a subroutine and get back. Recursion is allowed (it was not in the original BASIC version III).
- `LET`: (required) for single assignments of variables and array elements.
- `END`: it must be unique and be positioned as the last statement, required.
- `INPUT`: enabled for multiple vars input, but unable to print input strings (e.g. `INPUT "Enter a number";N` won't work).
- `DATA/READ/RESTORE`: (only for numbers), with a maximum of 1280 values per program.
- `REM`: for comments; of course, `REMARK` is also accepted; dib accepts also the apostrophe as a comment in the middle of the line becuase of its praticity to cut out part of the code without canceling it, but it must be poited out that this feature would be set in version V of the Dartmouth BASIC in 1969, two years later.
- `DEFFN`: to define a function with one argument (e.g. `DEF FNX(A)=...` or zero arguments `DEF FNX=...`). Two or more arguments are not available.
- `PAGE`: clear terminal and simulate a new printing page.
- `STOP`: which acts as `END` but may be repeated anywhere.
- `DIM`: with one or two dimensions arrays, with `A..Z` names (`A0..Z9` names as array identifiers are not available).

Operative notes:

- Line numbers range is 1-99999
- Operators are the classic fab-four (`* / + -`) plus the powering (`^`, that can be types also as `**`)
- Relational operators are the classic six (`= <> <= >= < >`)
- A strictly correct syntax is necessary for the parser to work; so (and this is not bad at all) no freedom is left in leaving behind closing parentheses; no doubt: this enhances clarity and eases corrections.
- The powering operator may be followed by unary operators `-` and `+`; e.g.:

```
12^-1E-2
3.2^+4.2
A^-E1
```

are all allowed form; in any case, for a better reading, you may enclose the exponent into parenthesis:

```
12^(-1E-2)
3.2^(+4.2)
```

```
A^(-E1) fam T
```

- The statement `DIM A(N)` dimensions a unidimensional array, which is by default a vertical array; this easies calculations of matrices by vectors; if you ever want to dimension a `N`-elements horizontal array, do it through `DIM A(0,N-1)` with base index set to 0 or `DIM A(1,N)` with base index set to 1; notice though that `DIM A(N)` dimensions a vector, while `DIM A(0,N)` is not a vector but a special case of a matrix. You can mix their use in any type of calculation, anyway.
- Variables list in `INPUT` may be separated by commas or semicolons, and even mixed up; e.g.:

```
INPUT A,B,C or
INPUT A;B,C
```

- The argument of the statement `DEFFNX` (if present) must be a one-character variable, in the range `A..Z`, as in `DEFFNX(R)=...` (remembering that `X` and `x` are two distinct identifiers).

3.4.2. BASIC functions

The functions of the Dartmouth BASIC are:

- `ABS(X)` returns the absolute value of `X`
- `ATN(X)` returns the arc-tangent of `X`, the returned value is in radians
- `COS(X)` returns the cosine of `X` in radians
- `EXP(X)` returns the exponential of `X`
- `INT(X)` returns the integer part of `X`
- `LOG(X)` returns the natural logarithm of `X`
- `RND(X)` returns a random number
- `SGN(X)` returns the sign of `X`: -1 if negative, 1 if positive, zero if zero
- `SIN(X)` returns the sine of `X` in radians
- `SQR(X)` returns the square root of `X`
- `TAN(X)` returns the tangent of `X` in radians.

Operative notes:

- The trigonometric functions work in radians mode only.
- The `INT` functions does not operate as explained at page 39 of the 1964 Dartmouth BASIC manual for version II; the fact is that, as stated by Kurtz in [2], starting with version III, the `INT` function was changed to allow the usage of `INT(X+.5)` both for positive and negative numbers, so that `INT(12.9)` is 12 and `INT(-4.2)` is -5 (this is confirmed by Gottfried with the example #5.2 in [3]).
- The `RND` function, having no `RANDOMIZE` (to appear in next versions of BASIC), has been enhanced in dib in this way: if the argument of `RND` is zero, the returned values are completely deterministic, and the same for every instance of execution (this is exactly the way the Dartmouth BASIC worked); this helps in configuring a program which needs random number, with a fixed set.
  If the argument is not zero, a new seed is set based on the time of day, so that it's likely the returned values are not deterministic at all (rather pseudo-random). Invoking for instance `LET A=RND(1)` is equivalent to the call

```
RANDOMIZE
A = RND(0)
```

  From now on, in the same instance of the program, `RND(0)` can be used, to explore all the pseudo-random values on the new seed trace.

3.4.3. BASIC matricial statements

The core of the Dartmouth BASIC version III is the matricial calculus. Here are the functions that are availabe:

- Matrix algebraic calculus:

```
MAT A=B+C
MAT A=B-C
MAT A=B*C
```

- Generate a zero matrix, i.e. a matrix of zeroes:
  `MAT A=ZER`
  Matrix `A` must have been declared with `DIM` (or used outside of the `MAT` statements, and thus implicitly dimensioned as 10×10) and need not to be square. In case of redefinition of the matrix, the new dimensions can be specified afterwards:
  `MAT A=ZER(4,6)`
- Generate a matrix of ones:
  `MAT A=CON`
  Matrix `A` must have been declared with `DIM` and need not to be square. In case of redefinition of the matrix, the new dimensions can be specified afterwards:
  `MAT A=CON(2,2)`
- Transpose a matrix:
  `MAT A=TRN(B)`
  I matrix has dimensions $m \times n$, the matrix `A` after transposition will have dimensions $n \times m$.
- Invert a matrix:
  `MAT A=INV(B)`
  The inversion algorithm is based on the Gauss factorization with partial pivoting, an algorithm that satisfies the majority of cases, but if the matrix is *very* sparse, the result may be not precise. In any case, I couldn't find any problem so far. Let me know if you find a matrix that is badly inverted...
- Generate the identity matrix:
  `MAT A=IDN`
  The result is a matrix with ones on the diagonal and zero elsewhere. In case matrix `A` must be re-dimensioned, this can be done in the process specifying the square dimension:
  `MAT A=IDN(4)`
  Of course, an identity matrix is square.

  ☞ **Warning:** if you declare a matrix, say 10×10, and you re-dimension the matrix to 4×4, you cannot successively re-dimension to larger dimensions, because the space is resized. So that the following is forbidden:

```
10 DIM A(10,10)
20 MAT A = IDN(4)
30 MAT A = IDN(7)  <-- faulty line
```

- Print a matrix to screen.
  `MAT PRINT A`
- Input matrix values from `DATA` statements:
  `MAT READ A`
  Of course there must be `DATA` statements anywhere in the code...
- Multiplying a matrix by a constant
  `MAT A=(k)*B`
  `k` may be any valid mathematical expression fitting the line length.

Operative Notes:

- `A0..Z9` variables cannot be used into matrices.
- All matrices and vectors must have been declared with `DIM`, even if smaller in dimension than 10 in either side (or used outside of the `MAT` statements and implicitly dimensioned as 10×10); this is the prescription presented at page 31 of the September 1966 manual for version III: "*...each such matrix must be declared in a DIM statement*". I interpret this in an extensive way, because a matrix declared not using `DIM`, is nonetheless an existing matrix.
- The BASIC version III admitted things like `MAT A = B*A`, but they resulted in a nonsense (see page 32 of the 1966 manual); I believe that version III used to calculate `MAT A` using `A` itself and printed the wrong result, because the 'in-place' calculation contemporarily changed matrix `A`.
  `dib` acts a bit differently; it calculates the right matrix, because it uses a temporary matrix to hold the result, so `A` is changed only at the end;
- `MAT A=B` is not available. Use `MAT A=(1)*B` in its place.

- `MAT A=-B` is not available; in case you need such an operation, use `MAT A=(-1)*B` in its place.
- `MAT READ`, `ZER`, `CON`, `IDN` may redefine arrays (which, if greater in dimension than 10 in either side, must have already been instantiated with DIM or implicitly), but only to reduce the array size in either side. This re-dimensioning is implicit (no warning is issued).
- `MAT TRN` transposes argument matrix, and may be used also with vectors;
- `MAT INV` sets a system var, named `DET`, which contains the determinant; `DET` should not be called before a `MAT INV`, and if it contains zero after an inversion, the matrix is singular. `DET` retains its value until next `MAT INV` statement (it will be replaced by next determinant value) or end of program.
- As a means to understand how good the inversion of a matrix is, multiply the original matrix by its inverse; the closer are diagonal elements to 1 and the closer are all other elements to zero, the better is the inversion; in particular, I observed that if elements that should be zero are in the order of magnitude from 1E-3 to 1E-6, the inversion is not much accurate, but in most cases satisfactory. If the order of magnitude is smaller than 1E-7, the result is good. If order of magnitude is even smaller than 1E-10, no better result can be achieved by dib.
- As a final remark, I observe with pride that my inversion algorithm can solve what Valentin Albillo calls "Mean Matrices" (ill-conditioned and unstable matrices that the HP-71B pocket computer cannot completely resolve). See [8].

3.5. Error messages

dib is, at present, an interpreter, not a compiler. Error treatment thus cannot be the same of a compiler.

Basing on this simple fact, I had to reconsider the whole error treatment, since, while a compiler does multiple passes to compile, being one of the first the syntax check, and so being able to print a long listing of error messages, an interpreter should stop after the first error, because going on would mean either instantiating not allocated variables, or calling not existing addresses, or doing useless loops, or something equally bad.

I came up to the following decisions:

- set up a general error treatment system to stop at the first met error, but capable of reverting to a multiple error generator changing a flag only (use it at your risk!);
- use the same error strings of the original compiler, as exposed on [1];
- use an extended set of strings, partly derived from [5], and partly conceived by me, where the original compiler lacked proper messages, or to expand error information to some extent

These decisions forced me to recalibrate errors, discerning which one could safely let the program flow and which one had to be considered definitely a stopper. In the following paragraphs, errors recognized by dib and their own stop-code are reported.

If invoked with option `-c`, dib won't stop after meeting many errors (not all!), and will continue until a definitely bad thing has happened, writing in the meantime all the detected error messages. Errors which are considered dangerous for the computer memory, or which prevent dib from going on, will immediately stop the interpretation anyway. If invoked with option `--errors-list`, dib will print on standard output the complete list of these error codes and their messages.

3.5.1. Standard error messages

Standard messages are exactly the same of the original 1966 Dartmouth BASIC. In the following, an error is defined "stopper" if the error is big enough to prevent dib from going on.

`0. DIMENSION TOO LARGE [NOT USED]`
(stopper: no)
It occurred in Dartmouth BASIC when the size of an array was too large for the available storage. In practice, this cannot be replicated in dib, because the available memory is predetermined in `dib.h`, and can be expanded at need (of course, depending on your system memory).

`1. ILLEGAL CONSTANT`
(stopper: no)
It occurs when there's an incorrect form in a number:

- more than 9 digits;
- other than [+-.0123456789E] in a number.

These conditions cannot fully be observed by dib, which makes use of the `strtod()` C function, which has higher capabilities than Dartmouth BASIC's original parser. In practice, you'll rarely meet this error, because `strtod()` is very very clever.

`2. ILLEGAL FORMULA`
(stopper: no)
It occurs in case of:

- missing (closing) parentheses;
- illegal variable names;
- missing multiplication operators (implicit multiplication is forbidden);
- illegal numbers

In the original Dartmouth System *illegal numbers* might not refer to the preceding error 1; probably it occurred during execution, in case of overflow; if it's so, I cannot replicate it, due to the way gcc treats overflow.

`3. ILLEGAL RELATION [NOT USED]`
(stopper: no)
It occurred in the Dartmouth system, when an invalid relational expression between `IF` and `THEN` was found. There are some problems in catching this error for dib, because the parser always interpret as wrong numbers what it cannot understand. For example, the following code fragment:

```
10 PRINT 3<<4
20 IF A=>6 THEN 10
```

causes dib to emit the `ILLEGAL FORMULA` error message at lines 10 and 20, rather than this one, because it interprets the first as `3 <` followed by `<4`, which is an illegal constant, and the second as `A =` followed by `>6`, which is another illegal constant. So this error message is never issued.

`4. ILLEGAL LINE NUMBER`
(stopper: yes)
It occurs when:

- the line number is not available or obtainable
- the line number is greater than 99999

The second is detected as a superimposed limitation, because the effective memory size may be increased at pleasure for both (to a certain extent).

`5. ILLEGAL INSTRUCTION`
(stopper: no)
It occurs when there's some illegal instruction (not a valid BASIC statement) after the line number or in a formula or in an expression. If you're familiar with Commodore BASIC, you'll remember the ubiquitous `?SYNTAX ERROR`; it's the exact correspondent.

`6. ILLEGAL VARIABLE`
(stopper: no)
It occurs when an illegal variable is detected, for instance, a two-letters variable name has been used, like `AB`.

`7. INCORRECT FORMAT`
(stopper: no)
It occurs when a wrong statement has been written in `FOR..NEXT..STEP` or `IF..THEN` structures.

`8. END IS NOT LAST`
(stopper: yes)
It occurs when:

- `END` is not in the last line;
- there's more than one `END` statement.

`9. NO END INSTRUCTION`

(stopper: yes/no)
Self explanatory.

`10. NO DATA`
(stopper: no)
It occurs when:

- there's a `READ` without `DATA`.
- a `RESTORE` is called without `DATA` statements.

It's perfectly legal instead having `DATA` with no `READ/RESTORE`.

`11. UNDEFINED FUNCTION`
(stopper: no)
It occurs when `FNX is called without defining DEF FNX` first. A `DEF FNX` definition may appear anywhere (it's caught before execution), but it *must* appear somewhere, in order for `FNX` to find what to calculate.

`12. UNDEFINED NUMBER`
(stopper: yes)
It occurs when `GOSUB`, `GOTO` or `THEN` are given a non existent destination line number.

`13. PROGRAM TOO LONG [NOT USED]`
(stopper: no)
In Dartmouth BASIC, at least in the first two versions, there was a limited number of program lines which were available. When a program reached that length, this error message appeared, maybe in conjunction with error 19 (see ahead). `dib` uses a huge list of pointers to string lines, whose index is the correspondent line number, so this error is meaningless and never issued.

`14. TOO MUCH DATA`
(stopper: yes)
It occurs when there are more values for `DATA` than those available (this limit is 1280 and is resizable).

`15. TOO MANY LABELS [NOT USED]`
(stopper: no)
It occurred in Dartmouth BASIC when there were too many strings in the program (due to memory limitations). I cannot use it, I don't want to use it!

`16. TOO MANY LOOPS`
(stopper: yes)
It occurs when there are too nested `FOR..NEXT` loops than those available (this limit is set to 26). Of course, you can use even thousands of `FOR..NEXT` loops, if they are not nested!

`17. NOT MATCH WITH FOR`
(stopper: no)
It occurs when:

- there's a `NEXT` with a different variable than that used within `FOR`
- there's a wrong `NEXT` nesting. e.g.:

```
FOR A...
   FOR B...
NEXT A
   NEXT B
```

The correct one is of course:

```
FOR A...
   FOR B...
   NEXT B
NEXT A
```

`18. FOR WITHOUT NEXT`
(stopper: no)

The program ends and no NEXT was found.

19. CUT PROGRAMS OR DIMS [NOT USED]
(stopper: no)
This error (which looks rather a warning) has some meaning only with tiny resources. Used in Dartmouth BASIC and not in dib.

20. SUBSCRIPT ERROR
(stopper: yes)
A non existent array index has been called. It may even refer to differences in a DIM declaration or peculiar usages with MAT statements.

21. ILLEGAL RETURN
(stopper: no)
It occurs when a RETURN, unrelated to any GOSUB, is found.

22. OUT OF DATA
It occurs when READ cannot read DATA values anymore. As shown at page 7 of the 1966 manual, this causes the program to stop, but it's not necessarily an error. In this sense, it is an advice that your program is ended. In version II [1] this message didn't even appear.

41. NEARLY SINGULAR MATRIX
(stopper: yes) This error message appears when either the Gauss factorization is not possible, because the matrix is singular or nearly so, or because the determinant is null (again because the matrix is singular). In either case the program stops.

42. DIMENSION ERROR
(stopper: yes)
This error signals an inconsistency in one of the MAT statements. Probably two matrices have not the right order in a multiplication, or there is a mistaken matrix identifier. In either case, the program stops.

3.5.2. Warnings

The following warnings are meant to report odd situations, and don't generally stop the program flow; they don't all appear in the original 1966 BASIC manual, but I reputed all of them meaningful in a programming environment. Most of them are retrieved from [12], (confirmed by the compiler assembler source), some from the DTSS simulator by T. Kurtz [13], and some have been invented by me.

23. SQUARE ROOT OF NEGATIVE NUMBER
24. LOG OF NEGATIVE NUMBER
25. LOG OF ZERO
26. ABSOLUTE VALUE RAISED TO POWER
27. DIVISION BY ZERO
These warnings are shown when you execute an illegal math operation; things are 'adjusted' and the program flow is not interrupted. With 'adjusted' I mean:

- absolute value of a negative number is taken before performing a square root
- the logarithm of zero or a negative number returns MINFF
- a negative real cannot be raised to power, so its absolute value is taken
- a division by zero returns INFF (see p. 36 in [12]) if the dividend is positive and MINFF if the dividend is negative.

This behaviors reflect the original Dartmouth BASIC ones.

29. MISMATCHED DIMENSIONS
This error appears when you specify for an array more dimensions than those used to declare the array.

30. OVERFLOW
31. UNDERFLOW
The first warning appears when a number greater than INFF (or lower than MINFF) is met, either as itself or as a

result of a math operation; if so, the result is set to `INFF` (or `MINFF`). The second appears when the absolute value of a number is lower than `ZERO` (but not zero), that is the lower limit of this emulator, either as itself or as a result of a math operation; if so, the result is set to 0 and printed as `0.` (with a final dot).
I said *as a result of a math operation*; to this, I must add that I let the CPU perform all the intermediate calculations, so it's occasionally possible that these warnings don't get printed; for example, if you multiply 1E50*1E50*1E-50 in an expression, the first intermediate result is surely an overflow for dib, but not for a modern CPU; then, when next operation returns a value not overflowing dib's capacity, no warning is issued while evaluating this final result.

`32. ZERO TO A NEGATIVE POWER`
`33. ZERO TO THE ZERO POWER`
The first warning appears when you try to calculate 0 to the power of a negative number, and the result is set to `INFF` (which tends to simulate $1/(0^N)$ or $(1/0)^N$, with $N > 0$, both leading to infinite). The second appears when both base and exponent are zero, and the result is set to 1 (because the function x^x approaches 1 as x approaches 0 from the right - see [10]).

`34. GOSUBS NESTED TOO DEEPLY`
This warning appears when you go past the `GOSUB..RETURN` limit for inner subroutines; this limit is set to 80 in `dib.h`.
If you perform a recursion (i.e. you call a `GOSUB` into the `GOSUB` itself) dib will gently let you perform it, but at your risk; this is not the case of the original BASIC of 1966; from the Sept. 1966 manual, page 28: "*The user must be very careful not to write a program in which a GOSUB appears inside a subroutine which refers to one of the subroutines already entered. (Recursion is not allowed!)*". See also the program `RECURS` in the `OWN` directory.

`36. NO MATRIX INVERSION DONE YET`
Self explanatory. The determinant calculated after this warning (e.g. before a matrix inversion) is meaningless if you try to use it.

`37. IGNORING ONE-ELEMENT LIST OR TABLE`
If you dimension a one-element array (vector or matrix), using `DIM A(0)` or `DIM A(0,0)`, the interpreter won't see this as a matrix dimensioning, and will set an implicit 10 elements array or a 10×10 elements matrix, the first time you use a reference to the element; this happens only with option base 0 and you won't lose the object: only be conscious you have a whole array, not just a one-element array. You'll never see this warning with option base 1 (option `-b`) if you try do dimension a one-element array with DIM A(1) or DIM A(1,1).

`43. EXP TOO LARGE`
The Dartmouth BASIC of 1966 had a limit for `EXP` set to 176.752531043 (which is the natural logarithm of `INFF`); if the exponent of `EXP` is greater than this limit, the result is set to `INFF` and the calculation goes on.

3.5.3. System messages

Sometimes, things go in such way dib cannot manage them. In such cases, errors are printed in lower letters, preceded by the identifier "`dib:`", and the program inevitably stops. Here's a list of all the system errors:

```
dib: out of memory for arrays.
dib: out of memory for DEF.
dib: file not found.
dib: unable to close file.
dib: line <> is beyond domain.
dib: out of memory for line
dib: cannot get a valid starting line.
dib: -<> is an illegal option.
dib: missing file name.
dib: out of memory for ultimate END.
```

These error messages are auto-explicative; the `<>` term matches with a value which is calculated or retrieved during execution, and incorporated into the message. You're likely to never see last system error message, unless you're OS has a very very tiny memory.

3.6. The examples

In the package there are many directories of examples:

- the directory named `1964/` gathers all the examples featured in the 1964 manual for version II of the Dartmouth BASIC. They are named according to the page number in which they appear. The output of those programs may not coincide with those of version II, since version III has slightly changed.
- the directory `GOTTFR/` gathers all the examples of the 1st edition of [3] which could be applied to dib enabling option `-b`, to set option base to 1 (the Pittsburgh machine on which these examples ran used a consistent index for arrays starting from 1);
- the directory `MATRIX/` contains various examples of matricial calculus;
- the directory named `OWN/` gathers all the specific examples I created for dib;
- the directory `CONTRIB/` (now empty) is aimed to gather all the contributions of dib's users. So step right up!

3.7. The vim syntax file

In the package you will find the `dib.vim` file. If you don't use vim, well, skip this paragraph and forget the file. If you use vim as your current editor, here's some instructions for installing the syntax for any dib file (only for UNIX users).

3.7.1. Installing dib.vim

Locate the `.vim` directory under your `$HOME` and find the file `filetype.vim". Add the two lines`

```
" dib (Dartmouth BASIC)
au BufNewFile,BufRead *.dib     setf dib
```

`(and create it if it doesn't exist); now step into .vim/syntax` and copy here the file `dib.vim`. Done.

3.7.2. Mapping the dib.vim syntax

Of course, you may want to use vim on files created with deer, so the extension is not present. In this case, I map the dib syntax within the vim/gvim resource file (for me `$HOME/.gvimrc`), adding the following line:

```
:map <F2> :set syn=dib<CR><Esc>
```

Of course you can map any key you want. From now on, load the file and hit F2: the colored syntax will activate.

3.7.3. Note on the dib.vim file

There is a note to say about the `dib.vim` file. The minus sign ahead of a number is seen as the negative sign if followed directly by a number; that is -24 is seen as -24 and colored accordingly. But if the number is inside an expression, like:

```
A-24
```

the minus follows the following highlighting rules:

1 it is colored like the following number if there's no blank space between the minus and the first digit, like in A-24

2 it is colored like an operator if there's a blank space between the minus and the first digit, like in A - 24

My advice is to separate the operators into an expression, and keep the minus tight to the number in case this expression or sub-expression (for instance, into parenthesis) begins with a negative number; in any case, note that the minus preceding a variable is always highlighted as an operator.

3.8. The differences with the Darmouth BASIC version III

Even if I tried to maintain all the intrinsic characteristics of version III while realizing dib, I couldn't (or didn't want to) make a perfect copy; the fact is dib operates on modern computers, is written in C and interprets BASIC sources, while Dartmouth BASIC (any version) operated on a GE mainframe (without monitors or mouses or windows), was written in machine language and compiled BASIC sources.

I expose here all the differences I found between dib (current version) and the Dartmouth BASIC III, with the prayer

to the occasional reader to send me all other topics I could leave off.

- if a calculation involves intermediate results greater than `INFF` of smaller than `MINFF`, calculation proceeds without interruption, since the overflow taken into account is not the Dartmouth BASIC one, but the gcc limit;
- the comment introduced by the apostrophe (’) in the middle of a code line was not in version III, but was made available since version V;
- the restrictions in inputting numbers at page 8 of the 1964 manual for version III is not observed by dib;
- the restriction at page 31 of the 1966 manual for version III ("*While the same matrix may appear in a MAT and an arithmetic statement, it must occur for the first time in a MAT statement...*") is not observed by dib;
- dib evaluates intrinsic booleans (for instance `IF A THEN` is admitted); I don’t know if Dartmouth BASIC could;
- dib accepts broken statements; again, I don’t know if Dartmouth BASIC could; the DTSS simulator by Kurtz [13] does, anyway;
- dib accepts lower characters in strings; the Dartmouth BASIC could not, I believe;
- dib considers lower-character variables and functions as different entities than corresponding upper-character ones; so `A1` and `a1` are two distinct variables, and `DEFFNA` and `DEFFNa` are two distinct functions; this feature was not in the original BASIC;
- the errors detection system is built on different needs; while the Dartmouth BASIC was a compiler, dib is an interpreter, and so some error messages and warnings may be issued in different places or times; besides, I had to invent some adding warnings and errors, to signal specific error conditions, not knowing which were (and even "if" they were) the right messages.
- The two enhancements on the variable `DET` (that holds the determinant of the matrix inversion, and helps in establishing if the matrix is singular), and on the function `RND(X)`, that includes the randomization process, were not, with high probabilities, on the original Dartmouth BASIC version III.

These differences, as you can see, won’t prevent you from writing correct and compatible programs shareable between dib and a (theoretical) Dartmouth BASIC III mainframe of 1966.

Can you hear the Beatles’ "Revolver" album playing? It’s 1966 again!

4. CONCLUSIONS

In the end, there are some people I want to thank, because their work made my job easier (or even possible).

The first one is **Diomidis Spinellis**, the author of `dds_basic`, the program which gave me the idea and offered me a frame on which basing my code. I learned a lot from his programming style, which is fine, essential, smart.

The second one is professor **Tao Pang**, from UNLV (University of Nevada, Las Vegas), for the book "*An Introduction to Computational Physics - 2nd edition*", Cambridge ed., 2006, a very useful manual for implementing a lot of scientific and mathematical routines and functions. I thank him again because he made available most of its software through the Internet, and indeed the first attempts for `matinvert()` were based on his `elgs()` and `migs()` functions, before implementing my own.

The third one is **Jack W. Crenshaw**, the author of the book "`Math Toolkit for real-time programming`", CPM Books, 2000, an invaluable mine of math tools, humor, style; he's a man with a vivid intelligence, capable of getting into the harder algorithm and turn it into a clear and easy tool. Every programmer should read this book. And figure out: I bought it by chance!

Last but not least is my wife, **Anna**, who supported my work and put up with my absence while I was holed up in my study trying to write working code; she complained now and then, but she loves me enough to tolerate it...

*****

I cannot leave without remembering with love professor **J.G. Kemeny** (in memory) and professor **T.E. Kurtz**, who created the BASIC programming language many years ago, an easy and lightweight language, suitable for mathematical and financial subjects, which guaranteed me an entry point in computer science. Thanks and thanks again.

*****

Finally, I propose this new acronym for BASIC:

Broad **A**nalysis with **S**imple and **I**ntuitive **C**oding

I hope you like dib.

5. BIBLIOGRAPHY

The following documents and references played an important role during the design and the coding of dib:

[1] "*Dartmouth BASIC manual*" for version II, October 1964, available as a pdf file at `https://www.bitsavers.org/pdf/dartmouth/dtss/196410_BASIC.pdf`

[2] "*BASIC Sessions*", by Thomas E. Kurtz, chapter XI of the book "History of programming languages", about the original BASIC environment, 1977 ca, available as a pdf file of a typewriter transcript (titled "BASIC") at `https://dl.acm.org/doi/10.1145/960118.808376`
A pdf file of chapter XI of the issued volume (which contains the same text) may be obtained through a free subscription at the acm.org portal.

[3] "*Programming with BASIC*", by Byron S. Gottfried, edition 1, 1975 (Italian version, as "*Programmare in BASIC*", Schaum 1982). This book apparently uses version V of BASIC, in a slightly different flavour, because in it executed in Pittsburgh (not Dartmouth) and because it's a DEC-10 machine.
The examples report the date February 1973; since Version V dates 1969, it not strange to think that it was implemented in 1971-72, while Dartmouth was issuing its version V.

[4] "*Commodore 64 C= - Reference Guide for the Programmer*", 1983 (Italian version, as "*Commodore 64 C= - Guida di riferimento per il programmatore*"), Commodore edition.

[5] Listing of the source code for version II of the Dartmouth BASIC compiler available as a pdf file at `https://web.archive.org/web/20070928081111/http://www.dtss.org/scans//BASIC/BASIC%20Compiler.pdf`.

[6] News about John Kemeny may be found at `http://cis-alumni.org/JKemeney.html`

[7] News about Thomas Kurtz may be found at `http://cis-alumni.org/TKurtz.html`

[8] "*Mean Matrices*" (DatafileVA014.pdf), an article by Valentin Albillo on ill-conditioned matrices is available as a pdf file at `https://www.hpmuseum.org/forum/thread-9090.html` (not sure it's still available)

[9] "*To Potential ALGOL Users*", Sept. 29, 1964, a note to Dartmouth ALGOL users by Stephen J. Garland, revised by D. W. Scott March 3, 1965, available as a pdf file at `https://www.bitsavers.org/pdf/dartmouth/dtss/196409_ALGOL.pdf`

[10] Su, Francis E., et al. "*Zero to the Zero Power.*" Mudd Math Fun Facts. `https://math.hmc.edu/funfacts/zero-to-the-zero-power/` where you can find an explanation to the fact that zero to the zero power is 1.

[11] Professor Bantchev, at the BASIC entry in `http://www.math.bas.bg/~bantchev/place/basic.html` (now discontinued) stated that the Original BASIC didn't treat boolean values, so that it's conceivable that a phrase like "`IF A THEN`" would have issued an error in Dartmouth BASIC.

[12] "*Dartmouth BASIC manual*" for version III, June 1965, Rev, September 1966 available as a pdf file at `https://swcomputingservices.com/hhhcdata/gebasicm.pdf`

[13] The DTSS simulator by Kurtz, along with documents about BASIC and ALGOL, and a lot of examples in both languages, my be downloaded at `https://dtss.dartmouth.edu/`

*****

The draft of this document begun in textual form on July 2008, while coding started on February 2, 2008. The pdf rendition was completed in June 2026.

I hope you like this program and appreciate the work I've done on it. Feedback is anyway greatly expected, and your contributions too. Send any message (greetings, critics, bugs, suggestions, programs and anything you think important) to the address below.

I am Antonio Maschio, and you can reach me at <ing dot antonio dot maschio at gmail dot com>.

Enjoy! It's GPL!

Table of Contents